Senior AI Engineering Curriculum

Day 2 · Topic 3: Streaming, OpenAI-Compatible APIs & Model Format Reference

Week 1 — Local Inferencing Foundations

Goal: Implement SSE token streaming with HuggingFace and Ollama, measure TTFT, consume a streaming endpoint via the OpenAI SDK, and understand every major LLM model format — which runtimes support which, and how to choose.

PART A — Response Streaming

What is Token Streaming?

Without streaming the client waits until the entire response is generated before receiving any output. For a 200-token response at 20 tok/s that is a 10-second blank screen. Streaming sends each token to the client the instant it is produced, giving the illusion of a thinking assistant and dramatically improving perceived responsiveness.

The two metrics that define streaming UX are:
• **TTFT (Time-To-First-Token)** — wall-clock time from request send to first token received. Dominated by prompt processing and model load time. Target: under 1 s for good UX.
• **TBT (Time-Between-Tokens)** — interval between consecutive tokens = 1 / tok_per_s. Dominated by model size, hardware, and quantisation. Target: under 100 ms (10+ tok/s).

How SSE Streaming Works

HTTP/1.1 streaming keeps the TCP connection open after the server starts responding. Each generated token is serialised to JSON and flushed immediately. The client reads chunks line-by-line as they arrive. Both Ollama's native API and the OpenAI API use newline-delimited JSON (NDJSON) — one JSON object per line. The OpenAI format prefixes each line with *data:* and ends with *data: [DONE]*.

Step	Who	What happens
1	Client	POST /api/generate {stream: true}
2	Server	Processes prompt tokens (prefill phase) — TTFT clock starts here
3	Server	Generates token 1, serialises to JSON, flushes over TCP
4	Client	Receives first chunk, renders token — TTFT recorded here
5	Server	Generates tokens 2 through N, flushes each immediately (TBT = interval)
6	Server	Sends {done: true} or data: [DONE], closes stream

Practical Implementation — Streaming

A.1 HuggingFace — TextStreamer (simple, blocking)

```
from transformers import AutoTokenizer, AutoModelForCausalLM, TextStreamer
import torch, time

MODEL_ID  = "gpt2"
tokenizer = AutoTokenizer.from_pretrained(MODEL_ID)
model     = AutoModelForCausalLM.from_pretrained(MODEL_ID, torch_dtype=torch.float32)
model.eval()

prompt  = "The most important thing about distributed systems is"
inputs  = tokenizer(prompt, return_tensors="pt")

# TextStreamer prints tokens to stdout as they are generated
streamer = TextStreamer(tokenizer, skip_prompt=True, skip_special_tokens=True)

start = time.perf_counter()
with torch.no_grad():
    output = model.generate(
        **inputs,
        max_new_tokens=80,
        streamer=streamer,
        do_sample=True,
        temperature=0.8,
        pad_token_id=tokenizer.eos_token_id,
    )
total_time = time.perf_counter() - start
new_tokens = output.shape[1] - inputs["input_ids"].shape[1]

print(f"\nTotal time : {total_time:.2f}s  |  {new_tokens / total_time:.1f} tok/s")
```

A.2 HuggingFace — TextIteratorStreamer (non-blocking, async-ready)

```
from transformers import AutoTokenizer, AutoModelForCausalLM, TextIteratorStreamer
import torch, threading, time

MODEL_ID  = "gpt2"
tokenizer = AutoTokenizer.from_pretrained(MODEL_ID)
model     = AutoModelForCausalLM.from_pretrained(MODEL_ID); model.eval()

prompt   = "The most important thing about distributed systems is"
inputs   = tokenizer(prompt, return_tensors="pt")
streamer = TextIteratorStreamer(tokenizer, skip_prompt=True, skip_special_tokens=True)

# Generate in background thread so main thread can consume tokens freely
thread = threading.Thread(target=model.generate, kwargs=dict(
    **inputs, max_new_tokens=80, streamer=streamer,
    do_sample=True, temperature=0.8,
    pad_token_id=tokenizer.eos_token_id,
))
start = time.perf_counter(); thread.start()

ttft, token_count = None, 0
```

```
for token_text in streamer:         # yields each token as it is ready
    if ttft is None:
        ttft = time.perf_counter() - start
    print(token_text, end="", flush=True)
    token_count += 1

thread.join()
total = time.perf_counter() - start
tbt   = (total - ttft) / max(token_count - 1, 1) * 1000

print(f"\n\nTTFT       : {ttft:.3f} s")
print(f"TBT (avg)  : {tbt:.1f} ms")
print(f"Throughput : {token_count / total:.1f} tok/s")
```

A.3 Ollama — Streaming via REST

```
import requests, json, time, sys

def stream_ollama(prompt, model="llama3"):
    url     = "http://localhost:11434/api/generate"
    payload = {"model": model, "prompt": prompt, "stream": True,
               "options": {"temperature": 0.8, "num_predict": 80}}

    start, ttft, chunks = time.perf_counter(), None, []
    with requests.post(url, json=payload, stream=True) as resp:
        for raw in resp.iter_lines():
            if not raw: continue
            chunk = json.loads(raw)
            tok   = chunk.get("response", "")
            if tok and ttft is None:
                ttft = time.perf_counter() - start
            sys.stdout.write(tok); sys.stdout.flush()
            chunks.append(chunk)
            if chunk.get("done"): break

    last  = chunks[-1]
    total = time.perf_counter() - start
    return {
        "ttft_s"   : round(ttft, 3),
        "total_s"  : round(total, 3),
        "tok_per_s": round(last["eval_count"] / (last["eval_duration"]/1e9), 1),
    }

m = stream_ollama("Explain PagedAttention in two sentences.")
print(f"\n\nTTFT={m['ttft_s']}s  Total={m['total_s']}s  Speed={m['tok_per_s']} tok/s")
```

A.4 OpenAI SDK Streaming against Ollama

```
from openai import OpenAI   # pip install openai
import time

client = OpenAI(base_url="http://localhost:11434/v1", api_key="ollama")
prompt = "What are the three pillars of production LLM serving?"
start, ttft, text = time.perf_counter(), None, ""

with client.chat.completions.create(
```

```
    model="llama3",
    messages=[{"role": "user", "content": prompt}],
    stream=True, temperature=0.7, max_tokens=100,
) as stream:
    for chunk in stream:
        delta = chunk.choices[0].delta.content or ""
        if delta and ttft is None:
            ttft = time.perf_counter() - start
        text += delta
        print(delta, end="", flush=True)

print(f"\n\nTTFT={ttft:.3f}s  Total={time.perf_counter()-start:.2f}s")
```

Testing & Metrics — Streaming

TTFT Benchmark Script

```
import time, json, threading, requests
from transformers import AutoTokenizer, AutoModelForCausalLM, TextIteratorStreamer
import torch

PROMPT  = "Explain transformer self-attention in simple terms."
RESULTS = {}

# HuggingFace
tok = AutoTokenizer.from_pretrained("gpt2")
mdl = AutoModelForCausalLM.from_pretrained("gpt2"); mdl.eval()
inp = tok(PROMPT, return_tensors="pt")
stm = TextIteratorStreamer(tok, skip_prompt=True, skip_special_tokens=True)
thr = threading.Thread(target=mdl.generate, kwargs=dict(
    **inp, max_new_tokens=50, streamer=stm, do_sample=False,
    pad_token_id=tok.eos_token_id))
t0 = time.perf_counter(); thr.start()
next(iter(stm))
RESULTS["HuggingFace gpt2 (CPU)"] = round(time.perf_counter() - t0, 3)
thr.join()

# Ollama
payload = {"model": "phi3:mini", "prompt": PROMPT,
           "stream": True, "options": {"num_predict": 50, "temperature": 0}}
t0 = time.perf_counter()
with requests.post("http://localhost:11434/api/generate", json=payload, stream=True) as r:
    for line in r.iter_lines():
        if line and json.loads(line).get("response"):
            RESULTS["Ollama phi3:mini (CPU)"] = round(time.perf_counter() - t0, 3)
            break

print(f"{'Runtime':<35} {'TTFT (s)':>10}")
print("-" * 47)
for k, v in RESULTS.items():
    print(f"{k:<35} {v:>10.3f}")
```

Metric	Formula	Target
TTFT	t(first_token) - t(request_sent)	< 1s GPU / < 3s CPU

TBT	1 / tok_per_s x 1000 (ms)	< 100 ms (>10 tok/s)
E2E Latency	TTFT + (n_tokens x TBT)	Depends on length
Throughput	n_tokens / total_elapsed (tok/s)	>20 GPU / >5 CPU (7B)
Prompt prefill	prompt_eval_duration / 1e9 (Ollama)	Scales with prompt length

PART B — OpenAI-Compatible APIs

Why OpenAI Compatibility Matters

The OpenAI REST API has become the de-facto standard interface for LLMs. Hundreds of tools — LangChain, LlamaIndex, Cursor, Continue.dev, Vercel AI SDK — all speak the OpenAI protocol. By exposing /v1/chat/completions, local runtimes like Ollama let you swap GPT-4 for a local model with a single environment variable change.

Endpoint	Method	Purpose
/v1/models	GET	List available models
/v1/chat/completions	POST	Multi-turn chat (system/user/assistant roles)
/v1/completions	POST	Legacy single-prompt completion
/v1/embeddings	POST	Generate embedding vectors

Request / Response Schema

```
# POST /v1/chat/completions — Request body
{
  "model": "llama3",
  "messages": [
    {"role": "system",    "content": "You are a concise AI infrastructure expert."},
    {"role": "user",      "content": "What is PagedAttention?"}
  ],
  "temperature": 0.7,    # 0=deterministic, 1=creative
  "max_tokens":  200,
  "stream":      false,  # true for SSE token streaming
  "top_p":       0.9,
  "stop":        ["\n\n"]  # optional stop sequences
}

# Response
{
  "id": "chatcmpl-abc123",
  "object": "chat.completion",
  "model": "llama3",
  "choices": [{
    "index": 0,
    "message": {"role": "assistant", "content": "PagedAttention is ..."},
    "finish_reason": "stop"   # "stop" | "length" | "content_filter"
  }],
  "usage": {"prompt_tokens": 42, "completion_tokens": 87, "total_tokens": 129}
}
```

curl Test Against Ollama

```
# Non-streaming
curl http://localhost:11434/v1/chat/completions \
  -H "Content-Type: application/json" \
  -d '{"model":"llama3","messages":[{"role":"user","content":"What is vLLM?"}],"max_tokens":80}' \
```

```
  | python -m json.tool

# Streaming (SSE — watch tokens arrive)
curl http://localhost:11434/v1/chat/completions \
  -H "Content-Type: application/json" \
  -d '{"model":"llama3","messages":[{"role":"user","content":"What is vLLM?"}],"stream":true,"max_tokens":80}

# List available models
curl http://localhost:11434/v1/models | python -m json.tool
```

PART C — LLM Model Format Reference

Choosing the wrong format is one of the most common sources of confusion when deploying LLMs locally. This section covers every major format, its internals, and which runtimes support it.

Format 1: PyTorch Binary (.bin)

The original HuggingFace weight format. Weights are serialised with Python pickle via torch.save(). Large models are split into numbered shards. Precision is FP32 or FP16 — no built-in quantisation. **Security risk:** pickle can execute arbitrary code on load. Being phased out in favour of SafeTensors.

Property	Value
Extension	.bin / pytorch_model-XXXXX-of-YYYYY.bin
Serialiser	Python pickle (torch.save)
Precision	FP32, FP16, BF16
Quantisation	None built-in
Security	Unsafe — pickle remote code execution possible
Runtimes	HuggingFace Transformers, DeepSpeed
Status	Legacy — prefer SafeTensors for all new work

Format 2: SafeTensors (.safetensors)

Developed by HuggingFace as a safe, fast replacement for .bin. Uses a compact header (JSON metadata) followed by raw tensor data — no code execution possible. Supports **zero-copy memory-mapped loading**, meaning multi-shard models can begin inference before all weights are fully resident in RAM. This is the current default format on the HuggingFace Hub.

Property	Value
Extension	.safetensors
Serialiser	Custom binary (header JSON + raw bytes)
Precision	FP32, FP16, BF16, INT8
Quantisation	None built-in — use bitsandbytes / AWQ separately
Security	Safe — no code execution possible
Runtimes	HuggingFace Transformers, vLLM, SGLang, TGI
Status	Current standard for training and GPU serving

Format 3: GGUF (.gguf)

The native format for llama.cpp and Ollama. A single self-contained binary file that stores model architecture metadata, tokeniser vocabulary, and quantised weight tensors. Replaced the older GGML format in August

2023. The quantisation level is encoded directly in the filename (e.g. llama-3-8b.Q4_K_M.gguf).

GGUF Quantisation Levels — Cheat Sheet

Level	Bits/W	7B Size	Quality vs FP16	Recommended Use
Q2_K	~2.6	~2.8 GB	Significant loss	Extremely RAM-constrained
Q3_K_M	~3.4	~3.3 GB	Moderate loss	8 GB RAM laptops
Q4_0	4.0	~3.8 GB	Minor loss	Fast CPU, older hardware
Q4_K_M	~4.5	~4.1 GB	Very minor loss	DEFAULT — best balance
Q5_K_M	~5.7	~4.8 GB	Negligible loss	Higher quality local
Q6_K	~6.6	~5.5 GB	Near-lossless	Quality-sensitive tasks
Q8_0	8.0	~7.2 GB	Near-identical	Best GGUF quality
F16	16	~14 GB	Reference	GPU reference / benchmarks

Property	Value
Extension	.gguf
Serialiser	Custom binary (llama.cpp project)
Precision	Q2 through Q8, F16, F32 — built into the file
Security	Safe — no code execution
Runtimes	llama.cpp, Ollama, LM Studio, Jan, GPT4All
Status	Standard for CPU and consumer GPU inference

Format 4: GPTQ (.safetensors + quantize_config.json)

GPTQ quantises weights to INT4 or INT8 using second-order Hessian information — more accurate than naive rounding. Weights are stored as safetensors but with a quantize_config.json alongside. Requires a GPU — CPU inference is not supported. Commonly found on HuggingFace Hub under tags like TheBloke/Llama-2-7B-GPTQ.

Property	Value
Extension	.safetensors + quantize_config.json
Bits	INT4, INT8
7B VRAM	~4.5 GB (INT4)
CPU support	No — GPU only
Runtimes	AutoGPTQ, vLLM (gptq flag), HF Transformers + auto-gptq
Status	Widely used; being superseded by AWQ on quality metrics

Format 5: AWQ (.safetensors + AWQ config)

AWQ (Activation-Aware Weight Quantisation) identifies the ~1% of weights that most affect output quality — salient weights — and protects them from quantisation error via per-group scaling. All weights are INT4 but with FP16 scaling factors stored alongside. Consistently outperforms GPTQ at the same bit-width on reasoning tasks. This is the recommended INT4 format for GPU production serving (covered in depth on Day 4).

Property	Value
Extension	.safetensors + config.json (AWQ metadata)
Bits	INT4 with FP16 per-group scaling factors
7B VRAM	~4.0 GB
Quality	Best INT4 quality — near FP16 on most benchmarks
CPU support	No — GPU only
Runtimes	AutoAWQ, vLLM (native support), HF Transformers + autoawq
Status	Production standard for INT4 GPU inference

Format 6: EXL2 (ExLlamaV2 mixed-precision)

EXL2 is a mixed-precision format used by the ExLlamaV2 inference engine. Different layers can be quantised at different bit-widths (e.g. 3.0 to 8.0 bits per weight), optimising quality per VRAM budget more granularly than GPTQ or AWQ. Popular in the enthusiast community for pushing quality on consumer GPUs.

Property	Value
Bits	2.0 to 8.0 (mixed per-layer)
Runtimes	ExLlamaV2, TabbyAPI only
CPU support	No
Status	Niche — community use, not production standard

Format 7: ONNX (.onnx)

ONNX is a cross-framework model representation useful for edge deployment (ONNX Runtime on mobile/embedded) and non-NVIDIA hardware (Intel OpenVINO, AMD ROCm). HuggingFace models can be exported via the optimum library. Less common for large LLMs due to graph size limitations and slower iteration cycles.

Property	Value
Extension	.onnx
Runtimes	ONNX Runtime, OpenVINO, TensorRT (via import)
Status	Niche for LLMs — better suited for smaller models and edge devices

Master Format x Runtime Compatibility Table

Use this as your quick reference when choosing a format for a given runtime. Y = supported, N = not supported, ~ = partial or experimental.

Format	HF Transformers	vLLM	Ollama	llama.cpp	TGI	ExLlamaV2	ONNX RT
PyTorch .bin	Y (legacy)	Y	N	N	Y	N	N
SafeTensors	Y (default)	Y	N	N	Y	N	N
GGUF	~ llama-cpp-py	N	Y	Y	N	N	N
GPTQ INT4	Y + auto-gptq	Y	N	N	Y	N	N
AWQ INT4	Y + autoawq	Y native	N	N	Y	N	N
EXL2	N	N	N	N	N	Y	N
ONNX	~ optimum	N	N	N	N	N	Y

Decision Guide — Which Format Should I Use?

Situation	Recommended Format
Training or fine-tuning	SafeTensors FP16 / BF16
CPU-only local inference	GGUF Q4_K_M via Ollama or llama.cpp
GPU serving in production (vLLM / TGI)	SafeTensors FP16 or AWQ INT4
Low VRAM GPU (8 GB) on GPU server	AWQ INT4 or GPTQ INT4
Maximum quality on consumer GPU locally	GGUF Q8_0 or EXL2 via ExLlamaV2
Edge / mobile / Intel hardware	ONNX via optimum export
Default HuggingFace Hub model download	SafeTensors (auto-selected)

Convert SafeTensors to GGUF (Quick Reference)

```
# Step 1 — Clone llama.cpp and install Python deps
git clone https://github.com/ggerganov/llama.cpp
cd llama.cpp && pip install -r requirements.txt

# Step 2 — Download a model from HuggingFace Hub (SafeTensors)
huggingface-cli download meta-llama/Meta-Llama-3-8B --local-dir ./llama3-hf

# Step 3 — Convert to GGUF FP16 (lossless intermediate)
python convert_hf_to_gguf.py ./llama3-hf --outfile llama3-f16.gguf --outtype f16

# Step 4 — Quantise to Q4_K_M (the production default)
./llama-quantize llama3-f16.gguf llama3-Q4_K_M.gguf Q4_K_M

# Step 5 — Verify and compare sizes
ls -lh llama3-*.gguf
```

```
./llama-cli -m llama3-Q4_K_M.gguf -p "Hello" -n 20
```

Key Takeaways

• **Streaming is essential UX infrastructure** — always measure TTFT and TBT as separate metrics. TextIteratorStreamer is the right pattern for FastAPI and async servers.
• **OpenAI /v1/chat/completions** compatibility makes Ollama a drop-in local GPT-4 replacement for any tool in the ecosystem.
• **SafeTensors** = training and GPU serving. **GGUF** = CPU and consumer GPU. **AWQ** = best INT4 GPU quality. These three cover 95% of real-world use cases.
• **GGUF Q4_K_M** is the sweet spot: roughly 70% size reduction vs FP16 with negligible quality loss.
• Never hardcode format assumptions in application code — parameterise from config so you can swap formats without code changes.

Next topic: **Day 3 — GGUF / SafeTensors deep-dive**: convert a HF model to GGUF with llama.cpp, inspect file internals, benchmark CPU vs GPU, and map model size to GPU tier.